Compact Hybrid Language Modeling

DASE7506 MP1 Small Language Model Challenge

Huang Leyi Student ID 3036708725

Overview

The submitted predictor combines a small GPT with training-derived n-gram statistics and two causal caches. It achieves 1.60173 BPB on the complete test split, compared with 2.10126 for the supplied baseline, a reduction of 23.77%. The neural model retains the baseline architecture; the improvements come from continued training, checkpoint averaging and adaptive combination of complementary predictions.

Result	Measured value
Full-test BPB	1.601725733
Validation BPB	1.587333826
Neural parameters	1,088,256
CPU scoring relative to baseline	2.03 times
Peak evaluation RAM	1.66 GiB
Uncompressed inference assets	48.40 MiB

Evaluation protocol

All experiments use the supplied WikiText-2 text and fixed BPE tokenizer with 2,048 tokens. The evaluator scores independent windows of 256 targets, including the final short window. Every target except the first token of each split is scored once. BPB is the total negative log-base-2 probability divided by the entire raw UTF-8 byte count. Validation contains 376,599 targets and 1,148,007 bytes; test contains 428,405 targets and 1,292,013 bytes.

The final configuration was selected on validation and frozen before its complete-test evaluation. The original model, baseline configuration, tokenizer, data checks and evaluator are preserved. All neural weights and static statistics originate from the supplied training split. Earlier development versions were also tested; the current revision was selected using validation.

Experimental question

Can a compact model improve predictions by combining general context modeling, reliable local phrase statistics and repetition within the current window? The matched-target control separates the benefit of this combination from the benefit of additional exposure to training text. Fixed-weight ablations then isolate the statistical branch and its adaptive gates.

Predictor design

Neural and statistical components

The neural component has four Transformer blocks, width 128, four attention heads, learned positions and tied input and output embeddings. Its logits are divided by temperature 0.95 before softmax. The statistical component is a pruned, interpolated six-gram model using Kneser-Ney-style continuation counts and count-dependent discounts [1]. It backs off to shorter contexts when evidence is sparse.

For a context h, retained successors receive discounted probability max(c(h,w) − D(c), 0) / c(h). The remaining probability mass is assigned to the lower-order distribution. Pruned successors therefore contribute to backoff mass rather than being silently renormalized among retained entries. Orders two through six use successor caps of 2,048, 256, 32, 16 and 16; orders four and above require context total at least two. Sorted context keys and compact sparse arrays keep storage within the asset budget.

Adaptive mixing

Let pN be the neural distribution, pS the static statistical distribution and pE the exact-match cache. The predictor first forms pB = (1 − a)pN + apS, then pX = (1 − b)pB + bpE. The static gate a depends on statistical confidence and whether the two components agree on their most likely token. Eight confidence bins crossed with agreement give sixteen groups. The exact-cache gate b depends on the longest available match, with weights 0, 0.10, 0.30, 0.35 and 0.45 for match lengths zero through four.

The exact cache searches only earlier contexts inside the current input window. Matches of lengths one through four contribute weights 1, 4, 16 and 64 to their already observed successors. Longer repeated phrases can therefore receive more influence than isolated repeated tokens.

Continuous cache

The final step is p = (1 − g)pX + gpC. Following the continuous-cache idea [2], pC assigns mass to previously observed successors using similarity between hidden states. This implementation normalizes hidden vectors and uses exp(10 × cosine similarity), restricted to earlier positions. The gate is 0.05, 0.025, 0.025 or zero according to the confidence of pX, with thresholds 0.1, 0.3 and 0.6.

Causality and reset behavior

At position t, a cache entry from position s is eligible only when s < t, so its successor x(s+1) is already in the observed prefix. Both caches are computed independently for each example and window. The first position has no eligible history and receives no cache contribution. No cache state persists across calls. Static tables contain training statistics, while window caches contain only the current observed prefix.

Training and model selection

Training uses batches of 32 sequences with 256 targets per sequence, AdamW with weight decay 0.1 and gradient clipping at 1. The first stage reproduces the supplied 1,200-update baseline from random initialization. Continued training keeps the same neural architecture and restarts the optimizer at each stage. The final two stages use cosine learning-rate schedules with 100-update warmup and a floor of 15% of the peak rate.

Stage	Added targets	Seed	Peak rate
Baseline 1200 updates	9,830,400	17	Baseline recipe
Initial continuation	3,613,342	29	0.0004
Continuation 5200 updates	42,598,400	73	0.0004
Continuation 2400 updates	19,660,800	101	0.00015
One statistical fit	3,613,342	None	Counts

The first continuation consists of 441 full updates and a final update with 670 scored targets. After the 5,200-update stage, snapshots at updates 4,800, 5,000 and 5,200 are averaged equally. The next stage starts from that average; its final snapshots at updates 2,000, 2,200 and 2,400 are averaged equally again. Averaging combines weights from a single trajectory and adds no training exposure. The resulting neural ancestry processes 75,702,942 targets. Including one statistical fit gives 79,316,284 targets.

Selection procedure

Complete validation losses select snapshots, averaging windows, n-gram order and calibration. Static and exact-cache gates are fitted by five coordinate-search passes over weights from zero to one in increments of 0.05. Temperatures 0.85, 0.90, 0.95, 1.00 and 1.05 are considered. Continuous-cache sharpness is searched over 5, 10, 20, 40 and 80, with gate weights 0, 0.025, 0.05, 0.10, 0.15, 0.20 and 0.30.

A width-256 continuation was also evaluated. Under a shared calibration, its averaged candidate reached validation BPB 1.61032, while a longer-trained width-128 average reached 1.59633. The smaller model was retained under the tested recipes. This comparison does not establish that width alone is harmful because the training schedules and exposure differ.

The final fine-gated six-gram candidate scored 1.587334 on validation. A fine-gated five-gram candidate scored 1.587449 and a coarser six-gram candidate scored 1.587612. These are small differences; the evidence for the overall hybrid mechanism is stronger than the evidence for the final order and bin refinements.

Results and controlled comparisons

Predictor	Validation BPB	Test BPB
Initial baseline	2.07108	2.10126
Final neural weights only at T 1	1.77129	Not measured
Matched-target neural control at T 1.05	1.77036	Not measured
Final hybrid	1.58733	1.60173

Matched training exposure

The control starts from the same final averaged neural checkpoint and receives a further 3,613,342 neural training targets, exactly replacing the hybrid's one statistical pass. Its total exposure is therefore 79,316,284 targets, equal to the final hybrid. The extra training uses seed 103, peak learning rate 0.00005 and the exact partial final batch. Temperature calibration over 0.90, 0.95, 1.00, 1.05 and 1.10 gives its best validation result of 1.77036.

The hybrid improves validation BPB by 0.18303, or 10.34%, relative to this control. The comparison establishes an advantage at matched processed-target exposure. It does not match computation: gradient updates and counting have different costs, and the hybrid has larger inference assets. The control uses one continuation recipe rather than an exhaustive search over all pure-neural training strategies.

Ablation with fixed neural weights

All ablations use the final neural weights and the complete validation split. Removed components are not recalibrated, so the results measure their contribution to the submitted configuration rather than the best possible replacement system.

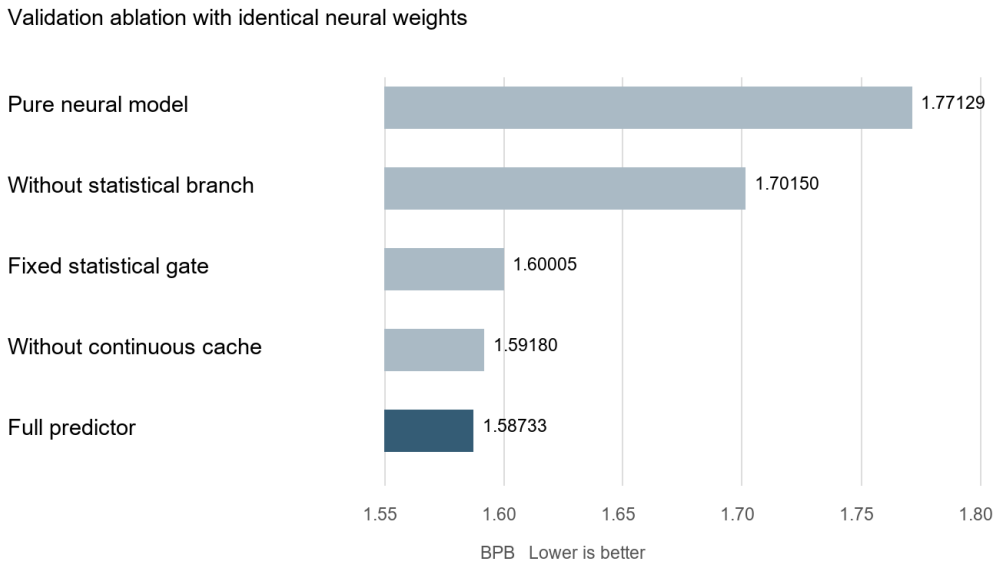


Figure 1 Fixed-weight validation ablation. The horizontal axis starts at 1.55 BPB.

Removing the static branch increases BPB to 1.70150. Replacing its adaptive gate with a fixed 0.6 mixture increases BPB to 1.60005. Removing the continuous cache increases BPB to 1.59180. These results indicate that the training-derived statistics provide the largest added contribution and that confidence-based mixing improves how those statistics are used.

Efficiency and verification

Measure	Observed	Limit
CPU scoring time	37.741 s	93.024 s
Relative scoring time	2.03 times	5 times
Peak evaluation RAM	1.660 GiB	4 GiB
Inference assets	48.400 MiB	64 MiB

Measurements use CPU FP32 with four PyTorch threads on the same Intel family 6 model 154 Windows machine. The paired baseline scoring time is 18.605 seconds. The table reports one complete test run per predictor, so timing variability is not estimated. Scoring excludes loading; the final process took 61.283 seconds including startup and loading. Peak resident memory is monitored for the evaluation process and children.

The frozen inference bundle contains the checkpoint, two implementation modules and the tokenizer provenance copy. Its assets total 50,751,473 bytes excluding the small manifest; including the manifest remains below 64 MiB. SHA256 hashes bind these files to the submitted score. The unchanged evaluator uses the supplied tokenizer and split files in code/data.

Correctness checks

Fifteen unit tests cover the model contract, compact statistics and both cache paths. Final checkpoint checks pass for finite outputs, normalization, future-token independence, batch independence, repeated calls and short prefixes. The maximum observed absolute log-normalization error is 8.94×10^{-7}. Hash verification confirms the nine checked fixed code and data files. The selected checkpoint is evaluated over all 428,405 test targets, including the short final window.

Interpretation and limitations

The components handle different evidence. Neural predictions capture broader context; sparse counts preserve phrase statistics; exact and continuous caches adapt to repetition in the current window. Adaptive gates reduce dependence on a weak component when evidence is limited. The same-weight fixed-mixture configuration scores 1.61188, compared with 1.58733 for the final configuration, so the gain is not explained solely by continued neural training.

The main trade-off is asset size: the static tables account for most of the bundle. Cache history is limited to the current 256-target window, and the system cannot exploit repetition across windows. Fine calibration adds validation choices and may overfit slightly; the five-gram and six-gram difference is too small to support a strong claim. The final test score is a measured result, while rankings and grades depend on course verification and assessment.

Reproduction and experiment costs

Extract the source archive and inference archive into one parent directory. The source includes the fixed data, evaluator, model implementations, training and calibration scripts, selected evidence and this report. Install Python 3.12, PyTorch 2.7.1 CPU and the pinned requirements. From code, run the unchanged evaluator with checkpoint ../inference_v3/checkpoint.pt, CPU, FP32, four threads and the test split. The repository README provides copyable installation, full training, calibration, ablation and evaluation commands. Direct evaluation requires no retraining or network access.

Final ancestry stage	Recorded seconds
Baseline training	740.849
Initial continuation	314.490
5200 update continuation	4132.894
2400 update continuation	1807.376
Statistical fit	33.554
Matched-target control additional training	332.903

The selected neural ancestry records approximately 1.94 hours across its four training stages, with stage validation included in continuation timings. This is not the total development cost. Broader searches include alternative count pruning, cache lengths, calibration, averaging windows, a wider-model trial and an untrained depth probe. COSTS.json inventories saved training and search runtimes; raw metadata supplies seeds, targets and checkpoint ancestry. Interrupted work and pauses mean these records do not determine total elapsed wall time. A documented earlier recovery adds at least 125 unsaved wide-model updates, or 1,024,000 targets, to search cost without changing selected checkpoint ancestry.

Acknowledgments

The course supplies the baseline GPT, original trainer, evaluator, WikiText-2 data and tokenizer. The submitted extensions implement compact smoothing, causal caches, calibration and continuation utilities. OpenAI Codex assisted with planning, implementation, experimental work and report drafting. No external training text or pretrained weights were used. Dataset redistribution notices remain in the supplied code README.

References

[1] Stanley F. Chen and Joshua Goodman. 1996. An Empirical Study of Smoothing Techniques for Language Modeling. ACL, pages 310–318. https://aclanthology.org/P96-1041/

[2] Edouard Grave, Armand Joulin and Nicolas Usunier. 2016. Improving Neural Language Models with a Continuous Cache. arXiv:1612.04426. https://arxiv.org/abs/1612.04426

[3] Stephen Merity, Caiming Xiong, James Bradbury and Richard Socher. 2016. Pointer Sentinel Mixture Models. arXiv:1609.07843. Dataset attribution follows the supplied package.